THE UNIVERSITY OF MELBOURNE

SWEN30006
Software Modelling and Design

Applying GoF Design Patterns

Part 2: ADAPTER, SINGLETON DESIGN PATTERNS

Revisit Factory pattern

The shift of focus (to patterns) will have a profound and enduring effect on the way we write programs.

—*Ward Cunningham* (*co-author of the Manifesto for Agile Software Development*) *and Ralph Johnson* (*co-author of Design Patterns: Elements of Reusable Object-Oriented Software*)

Ask your questions about Week 7 Lecture here.....

Nobody has responded yet.

Hang tight! Responses are coming in.

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Objectives

On completion of this topic, you should be able to:

- Apply some GoF design patterns
  - Façade : Covered in Week 7 Lecture
  - Decorator : Covered in Week 7 Lecture
  - Builder : Covered in Week 7 Lecture
  - Adapter : Topic of Discussion Today
  - Singleton : Topic of Discussion Today
  - (Concrete) Factory : Topic of Discussion Today (Factory Method Covered in SWEN20003)
  - Composite
  - Template, Strategy, Observer - revisited from SWEN20003
- Recognise GRASP principles as a generalization of other design patterns.
- We don't cover all GoF 23 design patterns in this course.

Problem 1: External Services with Varying Interfaces

- POS system needs to support several kinds of external services, including:
  - Tax calculators
    - e.g., TaxMaster, GoodAsGoldTaxPro
  - Accounting systems
    - e.g., SAP, GreatNorthern
  - Credit authorization services
- Each service has a different API, which cannot be changed.

Adapter (GoF)

***Problem**:*

- How to resolve incompatible interfaces, or provide a stable interface to similar components with different interfaces?

Solution (advice):

- Convert the original interface of a component into another interface, through an intermediate adapter object.
  - We *don't* mean programming language interfaces. E.g. Java interfaces.

Adapter (GoF)

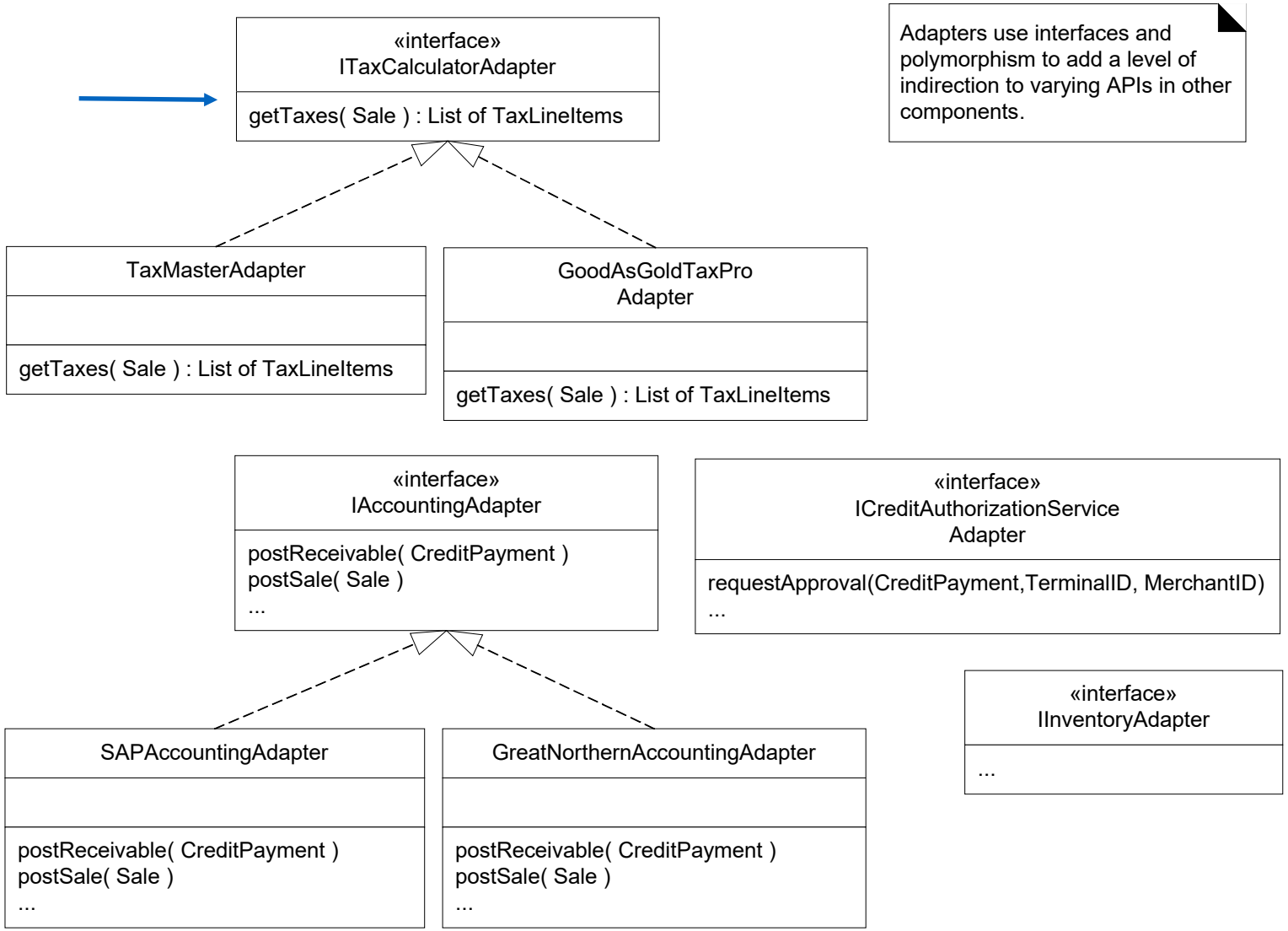
The Adapter Pattern converts the interface of a class into another interface that the client expects. Adapter lets classes work together, that may not have been able to work otherwise because of incompatible interfaces

1. Decouple client from interface implementation
2. (Interface Changes) encapsulated within the adapter

The Adapter Pattern

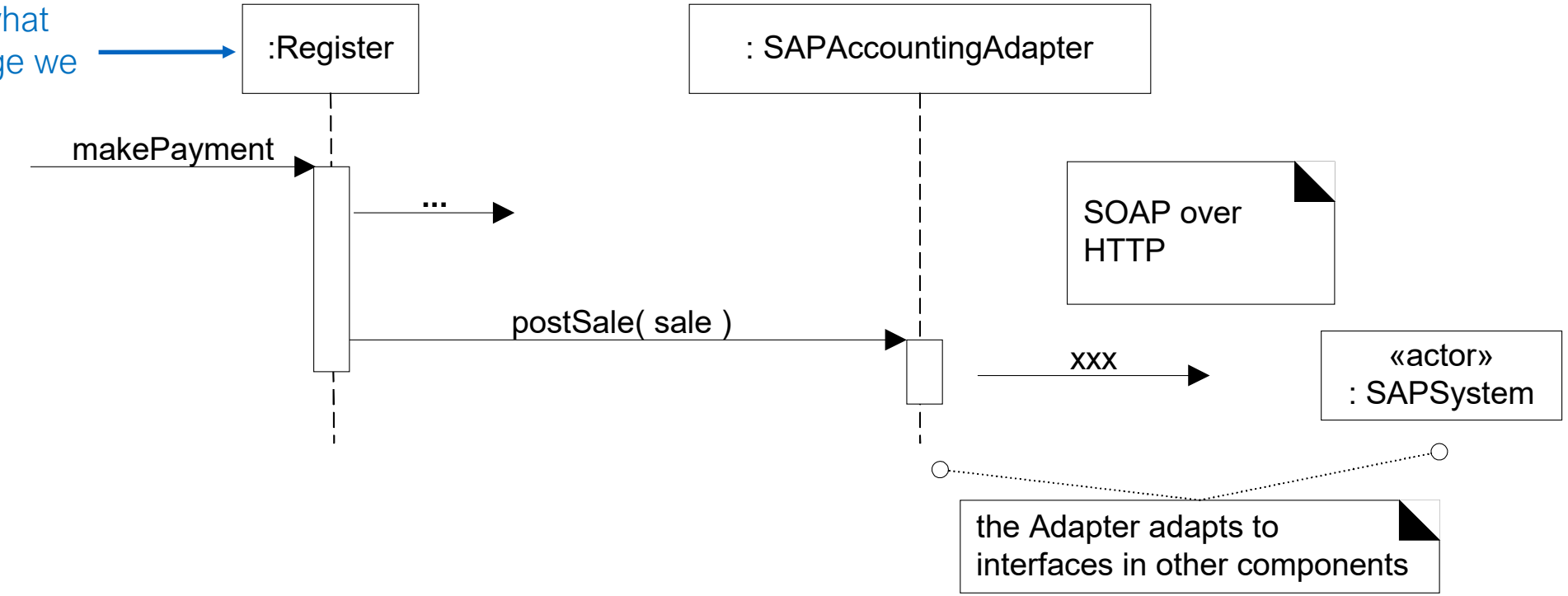
We define this interface.

For each external system we interact with, we define an adapter.



Using an Adapter

Doesn't know what external package we are using.



Example: Disney Madness

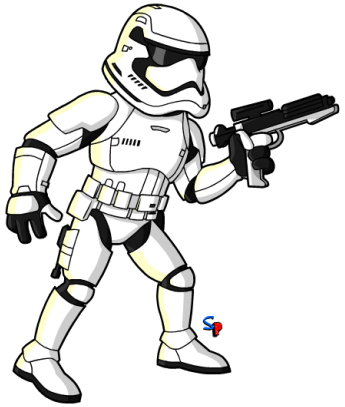
- Using an external libraries for Disney characters
  - APIs cannot be modified

These have different interfaces. We want to make these easier to use (with one general command).

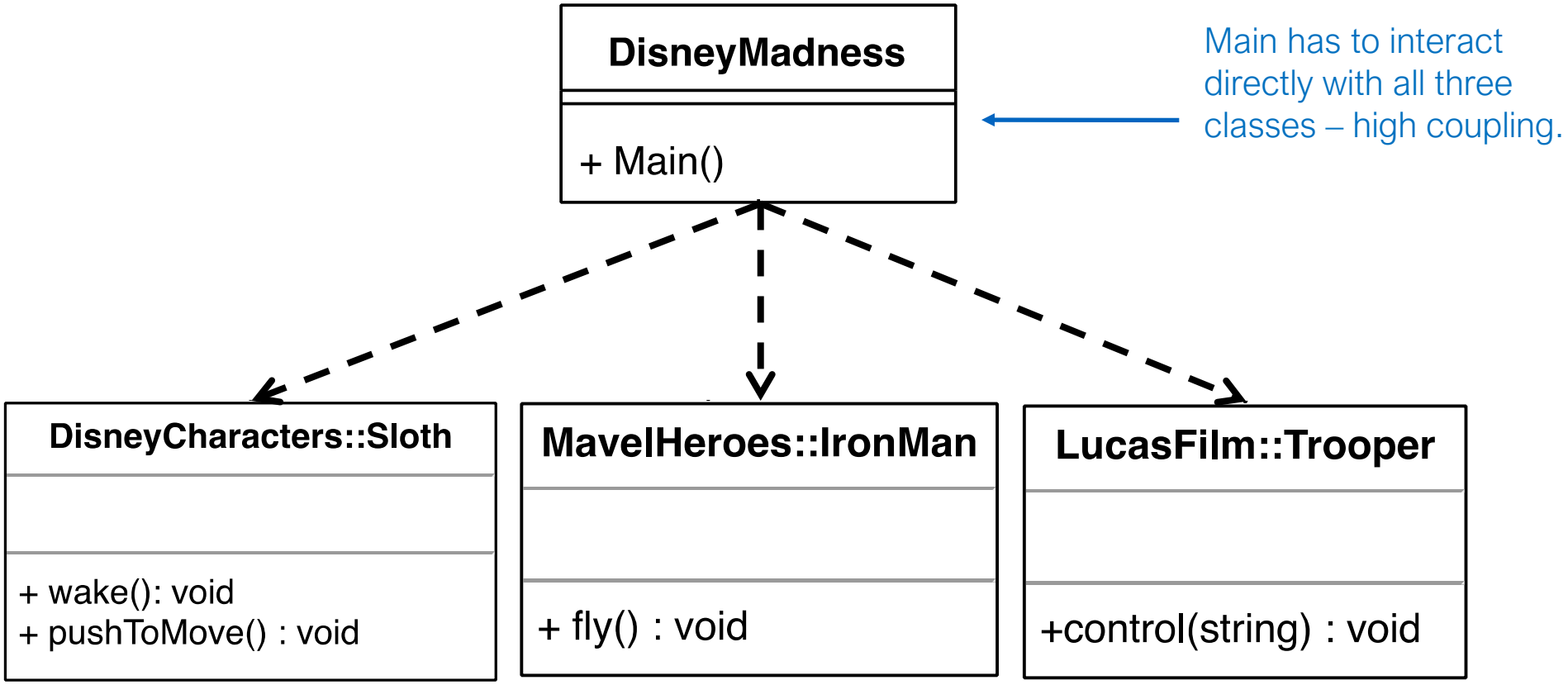
DisneyCharacters::Sloth
+ wake(): void + pushToMove() : void

MavelHeroes::IronMan
+ fly() : void

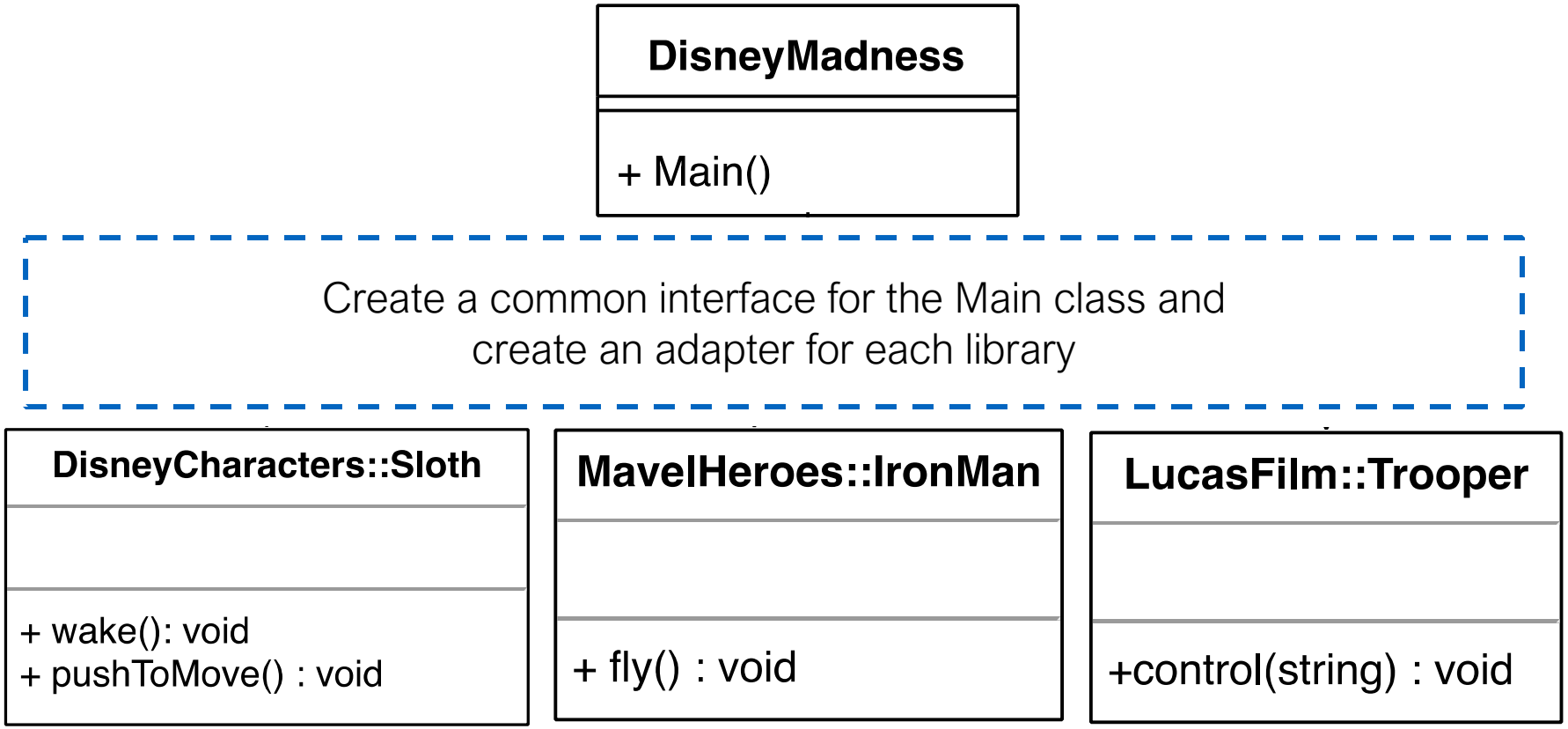
LucasFilm::Trooper
+control(string) : void



Disney Madness: Design *without* Adapter



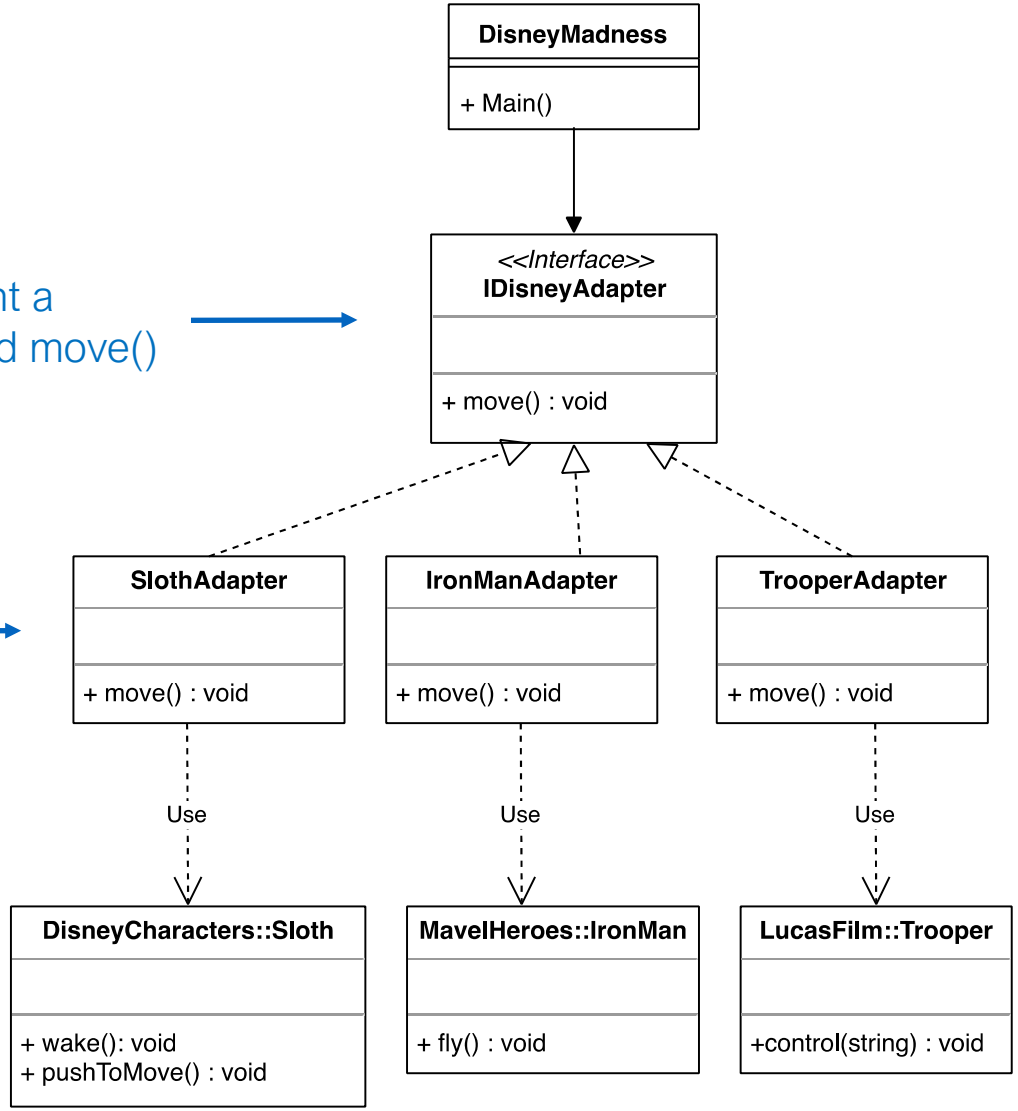
Disney Madness: Design *with* Adapter



Disney Madness: Design *with* Adapter

We implement a method called move()

Each character has an adapter which implements move()



Disney Madness: Implement *with* Adapter

```
public interface IDisneyAdapter {
   public void move();
}
```

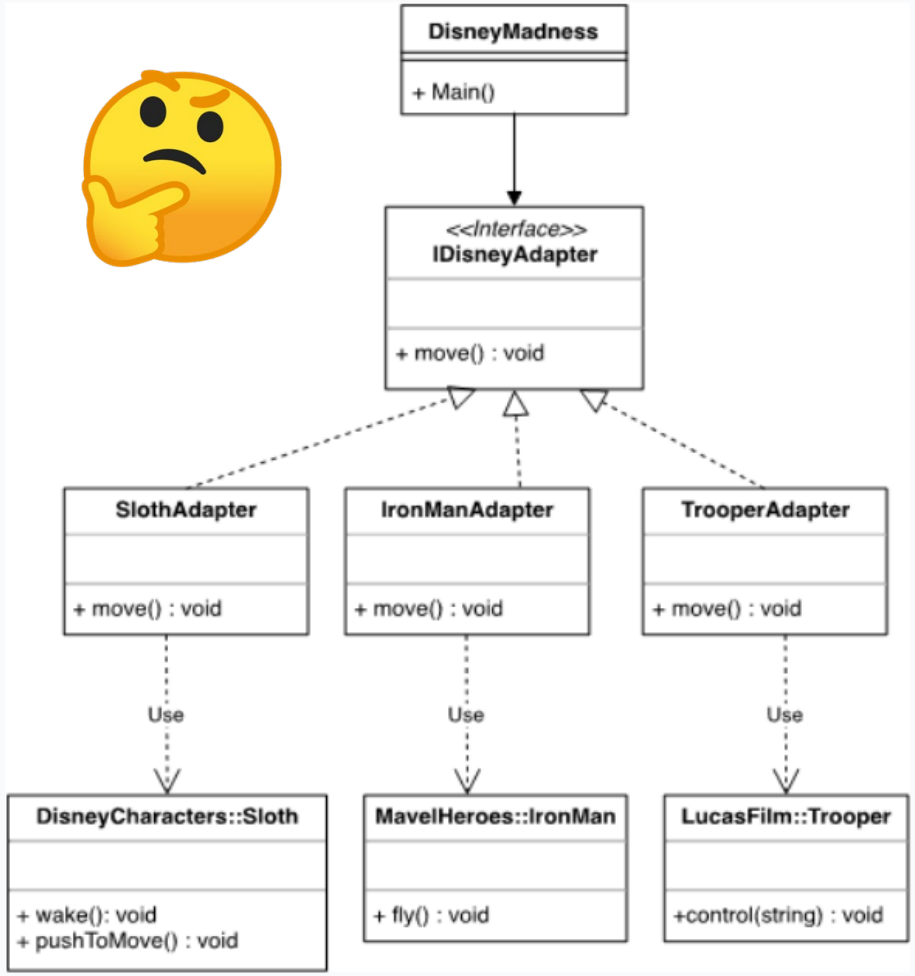
```
public class SlothAdapter implements IDisneyAdapter {
   private Sloth theSloth;
   public SlothAdapter(String name) {
     theSloth = new Sloth(name);
   }
   public void move() {
      theSloth.wake();
      theSloth.pushToClimb();
   }
}
```

Disney Madness: Implement *with* Adapter

```
public class DisneyMadess {
   public static void main(String[] args)  {
    //With adapter
    IDisneyAdapter slothAdt = new SlothAdapter("Flash");
    slothAdt.move();

    IDisneyAdapter ironManAdt = new  IronManAdapter("Tony");
    ironManAdt.move();

    IDisneyAdapter trooperAdt = new TrooperAdapter(”Storm");
    trooperAdt.move();
  }
}
```

Now the code is simpler.
We don’t need to know how each character moves. This is hidden away.

Which GRASP principle(s) are used by Adapter



Polymorphism 0%

Creator 0%

Indirection 0%

Pure Fabrication 0%

Information Expert 0%

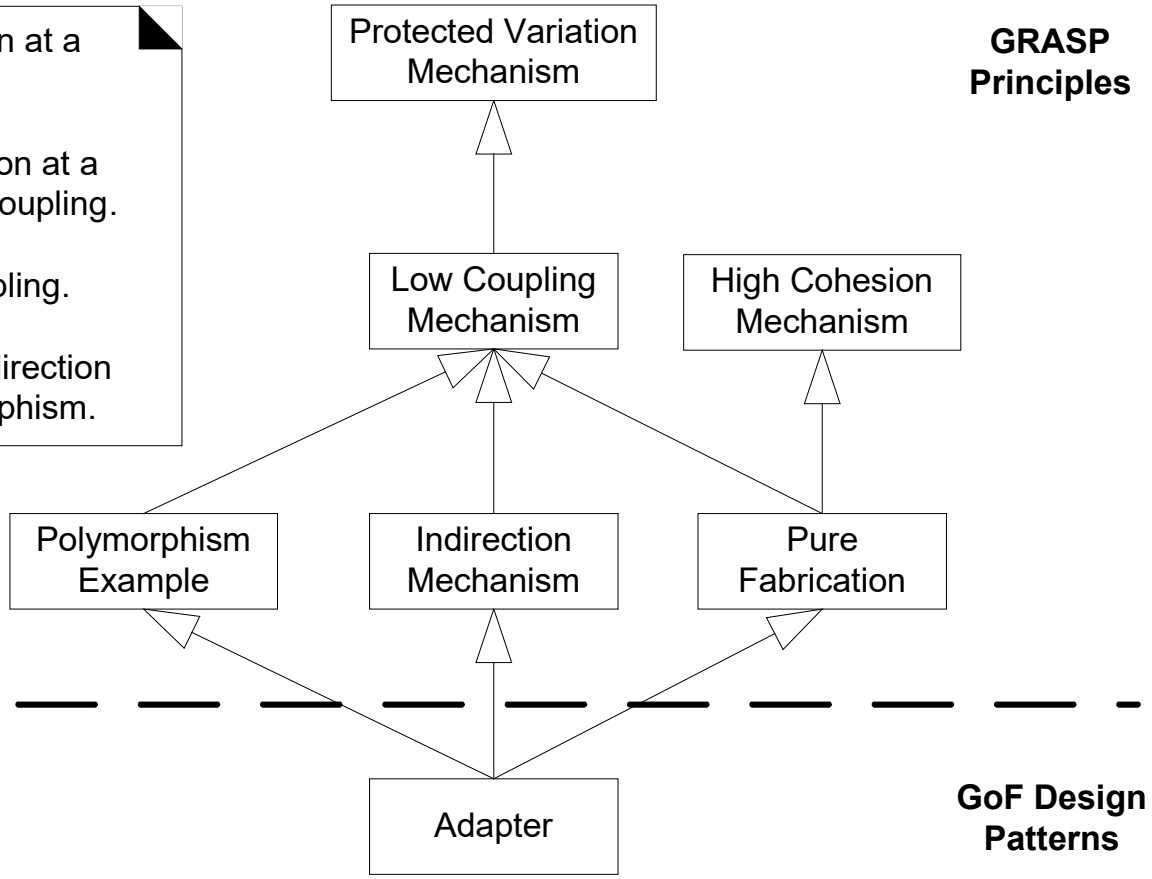
Adapter in relation to GRASP Principles

Low coupling is a way to achieve protection at a variation point.

Polymorphism is a way to achieve protection at a variation point, and a way to achieve low coupling.

An indirection is a way to achieve low coupling.

The Adapter design pattern is a kind of Indirection and a Pure Fabrication, that uses Polymorphism.



Ask your questions about Week 7 Lecture here.....

Nobody has responded yet.

Hang tight! Responses are coming in.

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Issues arising from Adapter

- Who creates the adapters?
- Which class of adapter should be created?
  - Should the Register object create SAPAccountingAdapter?

Issues arising from Adapter

- Who creates the adapters?
- Which class of adapter should be created?
  - Should the Register object create SAPAccountingAdapter?

Partial Answer:

- Separation of Concerns/High Cohesion
  - Creating adapters is not pure application logic
  - Adapter object is not a domain object
- **Use GRASP Pure Fabrication pattern**

Doesn't correspond to anything in real world.

So we fabricate an object to manage this.

Factory

Problem:

- Who should be responsible for creating objects when there are special considerations, such as complex creation logic, a desire to separate the creation responsibilities for better cohesion, and so forth?

Solution (advice):

- Create a Pure Fabrication object called a Factory that handles the creation.

(Concrete) Factory

Aka **Simple Factory**

- Simplified version of GoF Abstract Factory pattern

Advantages:

- Separate responsibility of complex creation into cohesive helper objects.
- Hide potentially complex creation logic.
- Allow introduction of performance-enhancing memory management strategies, such as object caching or recycling.
  - e.g. connecting to a database many times.

The Factory Pattern

The adapters we looked at before

ServicesFactory
accountingAdapter : IAccountingAdapter inventoryAdapter : IInventoryAdapter taxCalculatorAdapter : ITaxCalculatorAdapter
getAccountingAdapter() : IAccountingAdapter getInventoryAdapter() : IInventoryAdapter getTaxCalculatorAdapter() : ITaxCalculatorAdapter ...

note that the factory methods return objects typed to an interface rather than a class, so that the factory can return any implementation of the interface

Properties File Entry

```
! taxcalculator.class.name=TaxMasterAdaptor
taxcalculator.class.name=GoodAsGoldTaxProAdaptor
```

This uses "lazy creation" - don't create adapter unless required.

```
if ( taxCalculatorAdapter == null )
{
  // a reflective or data-driven approach to finding the right class: read it from an
  // external property

  String className = System.getProperty( "taxcalculator.class.name" );
  taxCalculatorAdapter = (ITaxCalculatorAdapter) Class.forName( className ).newInstance();

}
return taxCalculatorAdapter;
```

What makes the factory pattern is the ability to be able to get the adapter (or what the factory is making), and to get the right one. The means by which the right one is set can vary.

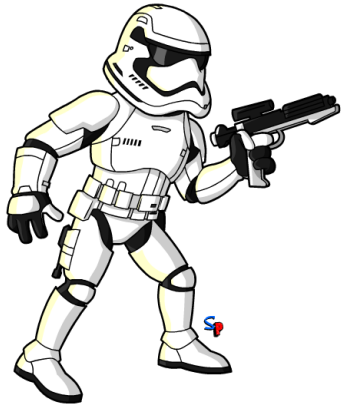
Example: Disney Madness

- Using an external libraries for Disney characters
  - APIs cannot be modified

DisneyCharacters::Sloth
+ wake(): void + pushToMove() : void

MavelHeroes::IronMan
+ fly() : void

LucasFilm::Trooper
+control(string) : void



Implement DisneyMadness *with* Adapter and Factory

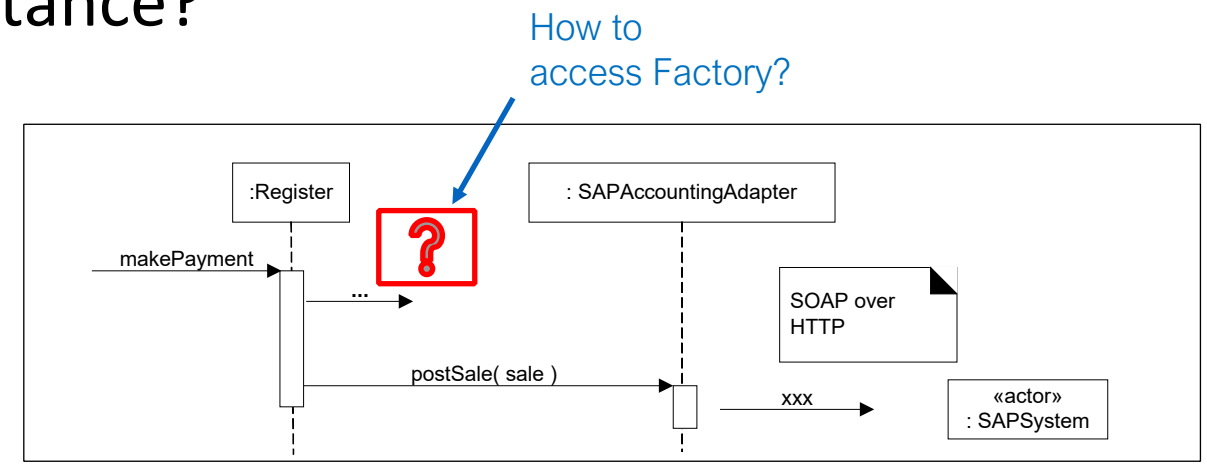
```
public class DisneyFactory {
   private IDisneyAdapter characterAdapter = null;

  /* Retrieve character adapter based on character name */
  public IDisneyAdapter getDisneyAdapter(String  character, String name) {
    if(character.equals("ironman")) {
      characterAdapter = (IDisneyAdapter) new IronManAdapter(name);
    } else if(character.equals("sloth")) {
      characterAdapter = (IDisneyAdapter) new SlothAdapter(name);
    } else if(character.equals("trooper")) {
      characterAdapter = (IDisneyAdapter) new TrooperAdapter(name);
    }
    return characterAdapter;
  }
}
```

We could use a method, rather than a variable in a properties file, to determine adapter type.

We could use an Enum above instead of a string.

Issues arising from Factory

- Who creates the Factory and how is it accessed?
  - It may need to be called from various places
- Oftentimes only one instance of this factory is needed.
- How to get visibility to this single **ServicesFactory** instance?



Issues arising from Factory

- How is Factory accessed?
  - It may need to be called from various places
- How to get visibility to this single ServicesFactory instance?

Partial Answer:

- Only one instance of the factory is needed
- Pass the factory to methods or initialise objects with a reference? *No. Adds a lot of complexity.*
- Use **Singleton pattern** to provide a single access point through global visibility

Singleton pattern is often used with Factory pattern.

Singleton (GoF)

Problem:

- Exactly one instance of a class is allowed—it is a "singleton." Objects need a global and single point of access.

Solution (advice):

- Define a static method of the class that returns the singleton.

The Singleton Pattern ensures a class has only one instance, and provides a global point of access to it.

Singleton Pattern in *ServicesFactory* Class

UML notation: this '1' can optionally be used to indicate that only one instance will be created (a singleton)

1

ServicesFactory

instance : ServicesFactory

accountingAdapter : IAccountingAdapter
inventoryAdapter : IInventoryAdapter
taxCalculatorAdapter : ITaxCalculatorAdapter

getInstance() : ServicesFactory

getAccountingAdapter() : IAccountingAdapter
getInventoryAdapter() : IInventoryAdapter
getTaxCalculatorAdapter() : ITaxCalculatorAdapter
...

UML notation: in a class box, an underlined attribute or method indicates a static (class level) member, rather than an instance member

singleton static attribute

singleton static method

```
// static method
public static synchronized ServicesFactory getInstance()
{
if ( instance == null )
   instance = new ServicesFactory()
return instance
}
```

Lazy Initialization

Same as we did before for Adapters.

Use syncronized modifer to avoid race conditions.

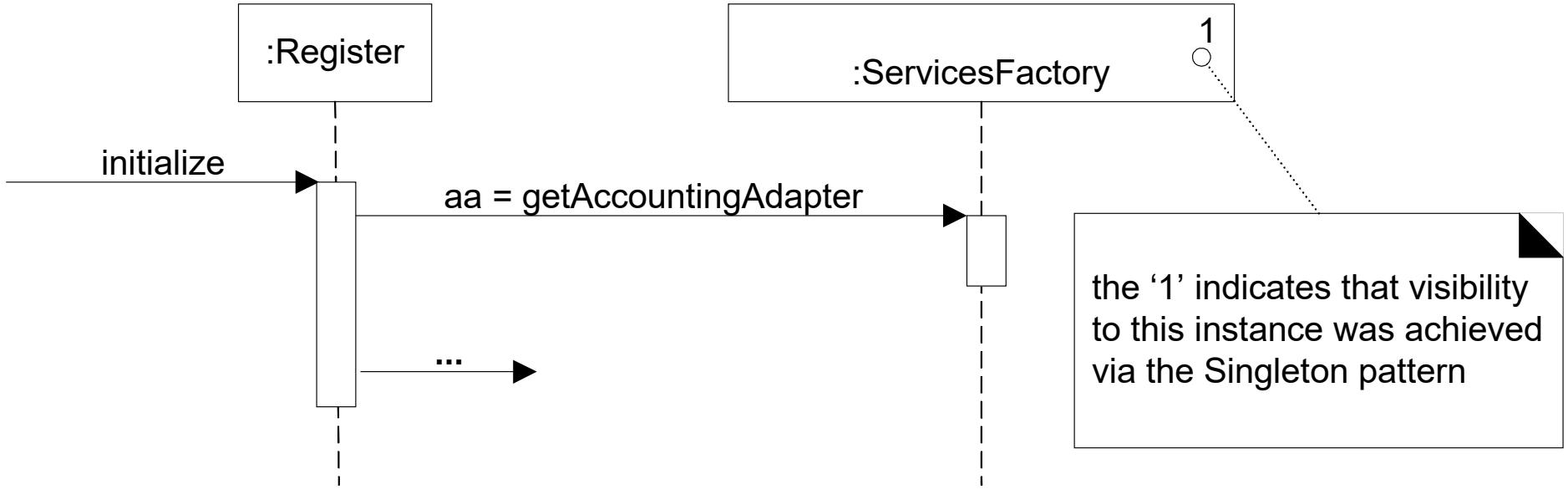
Accessing the Singleton Factory

```
public class Register
{
    public void initialize()
    {
        ... do some work …

        // accessing the singleton Factory via the getInstance call
        accountingAdapter =
              ServicesFactory.getInstance().getAccountingAdapter();

        ... do some work ...
    }
    // other methods...
} // end of class
```

Solving the problem from slide 31.

SingletonClass.getInstance() is globally visible

Implicit *getInstance* Singleton Pattern Message




```
aa = ServicesFactory.getInstance().getAccountingAdapter();
```

Static: why some and not all?

Why aren't all Singleton service methods *static?*

E.g. static getAccountingAdapter(): …

- May want subclasses: static methods are not polymorphic (virtual); no overriding in most languages
- Most remote communication mechanisms (e.g. Java's RMI) don't support remote-enabling of static methods.
- A class is not always a singleton in all application contexts.

Problem 1 Revisited: External Services with Varying Interfaces

Possible solution:

- "To handle this problem, let's use Adapters generated from a Singleton Factory."

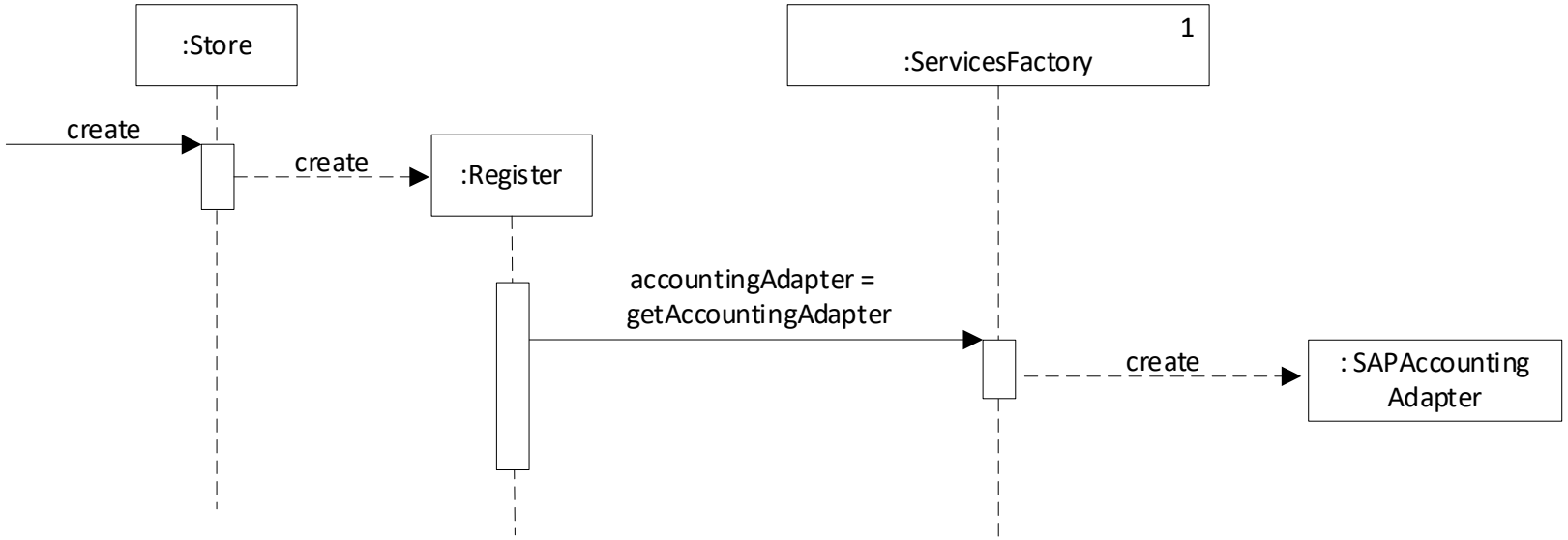
This is language we can use to talk to our colleagues in terms of patterns.
We can discuss solutions in a simpler way.

- Design reasoning based on:
  - Controller
  - Creator
  - Protected Variations
  - Low Coupling
  - High Cohesion
  - Indirection
  - Polymorphism
  - Adaptor
  - Factory
  - Singleton

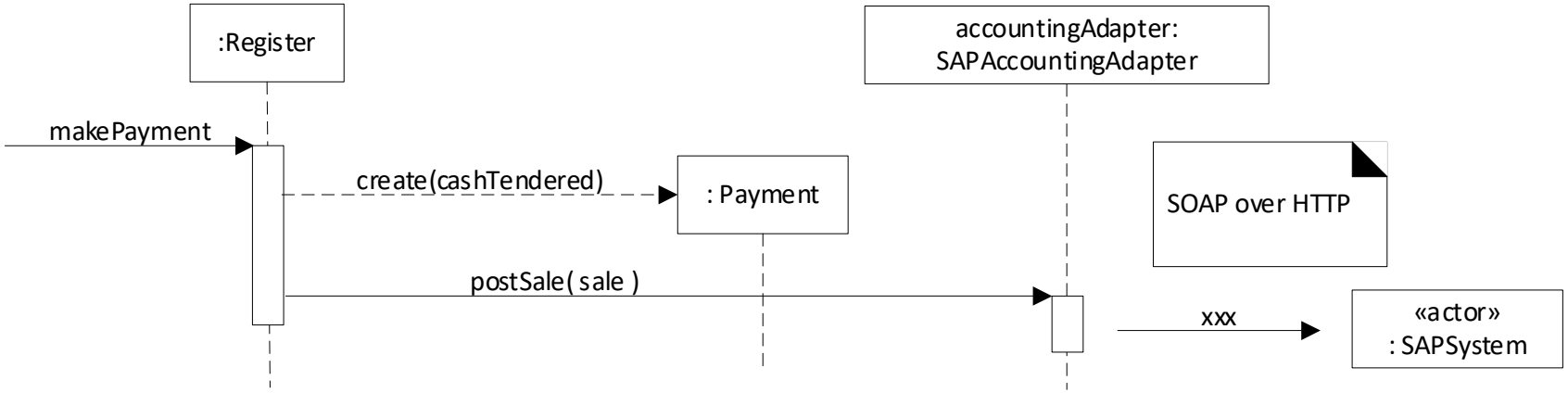
Register in POS example.

Adapter, Factory, and Singleton Patterns

Creating the Solution



Using the Solution



Ask your questions about Week 7 Lecture here.....

Nobody has responded yet.

Hang tight! Responses are coming in.

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

More examples, on YouTube

From LMS Design Pattern Resources:

- New Think Tank Design Patterns Tutorials
  - Adaptor Design Pattern
  - Factory Design Pattern
- Design Patterns in OO Programming: by C. Okhravi based on the textbook Head First Design Patterns
  - Factory Design Pattern
  - Singleton Design Pattern

Lecture Identification

Lecturer: Dr. Rajesh Chittor Sundaram

E-mail: c.s@unimelb.edu.au

Semester: S1 2025

© University of Melbourne 2025

These slides include materials from:

Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition, by Craig Larman, Pearson Education Inc., 2005.